



Open Ended Lab 11 - Worksheet

Name: Maleeha Prabha	ID: 09991 10292	Section: T1
----------------------	--------------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Design Module Codes for **Program Counter**, **PC Adder**, **Branch Adder**, **Mux2**, **immGen**.

Program Counter

`timescale 1ns / 1ps

//

// Company:

// Engineer:

//

// Create Date: 04/11/2026 10:57:33 AM

// Design Name:

// Module Name: ProgramCounter

// Project Name:

// Target Devices:

// Tool Versions:

// Description:

//

// Dependencies:

//

// Revision:

// Revision 0.01 - File Created

// Additional Comments:

//

//

module ProgramCounter(

input [31:0]next_PC,

input clk,

input reset,

output reg [31:0] PC

);

// wire [31:0]next_count;

[illegible]

```
// Create Date: 04/11/2026 11:39:15 AM
// Design Name:
// Module Name: immGen
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
```

```

module immGen(
    input [31:0] inst,
    output reg [31:0] imm
);

always @(*) begin
    case(inst[6:0]) //opcode
        //i-type
        7'b0000011, // lw
        7'b0010011: // addi, etc.
            imm={{20{inst[31]}}, inst[31:20]};
        //s-type
        7'b0100011:
            imm={{20{inst[31]}}, inst[31:25], inst[11:7]};
        //b-type
        7'b1100011: // beq, bne, etc.
            imm = {{19{inst[31]}}, inst[31], inst[7],
                inst[30:25], inst[11:8], 1'b0};
        default:
            imm = 32'b0;
    endcase
end

endmodule

```

Branch Adder

[illegible]



```
// Create Date: 04/11/2026 11:12:47 AM
// Design Name:
// Module Name: branchAdder
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////

module branchAdder(
    input[31:0] PC,
    input[31:0] imm,
    output[31:0] branch_tar
);
    assign branch_tar=PC +imm;
endmodule
```

Testbench:

```
`timescale 1ns / 1ps

module tb_Task1;

    reg    clk;
    reg    reset;
    reg    PCSrc;
    reg [31:0] inst;

    wire [31:0] PC;
    wire [31:0] upd_PC;
    wire [31:0] imm;
    wire [31:0] branch_target;
    wire [31:0] next_PC;

    ProgramCounter uPC (
```



```
.clk (clk),
.reset (reset),
.next_PC (next_PC),
.PC (PC)
);

pcAdder uPCAdder (
.PC (PC),
.upd_PC (upd_PC)
);

immGen uImmGen (
.inst (inst),
.imm (imm)
);

branchAdder uBranchAdder (
.PC (PC),
.imm (imm),
.branch_tar (branch_target)
);

MUX uMux (
.in0 (upd_PC),
.in1 (branch_target),
.select (PCSrc),
.out (next_PC)
);

initial clk = 0;
always #5 clk = ~clk;

initial begin
$monitor("[%0t ns] PC=%0d | PC+4=%0d | imm=%0d |
branch_target=%0d | next_PC=%0d | PCSrc=%0b",
$time, PC, upd_PC, $signed(imm), branch_target, next_PC, PCSrc);
end

initial begin
reset = 1;
PCSrc = 0;
inst = 32'b0;

@(posedge clk); #1;
reset = 0;
$display("\n[%0t ns] Reset released - PC should be 0", $time);
$display("\n--- TEST 1: Sequential PC (PCSrc=0) ---");
PCSrc = 0;
```



```
inst = 32'b0;

repeat(5) begin
    @(posedge clk); #1;
    $display(" PC=%0d | PC+4=%0d | next_PC=%0d", PC, upd_PC,
next_PC);
end

    $display("\n--- TEST 2: Branch taken (PCSrc=1) - BEQ 32'h00000463,
imm=8 ---");
    inst = 32'h00000463;
    PCSrc = 1;

    repeat(3) begin
        @(posedge clk); #1;
        $display(" PC=%0d | imm=%0d | branch_target=%0d | next_PC=%0d",
PC, $signed(imm), branch_target, next_PC);
    end

    $display("\n--- TEST 3: immGen I-type - ADDI x1,x0,-5 (32'hFFB00093) ---
");
    PCSrc = 0;
    inst = 32'hFFB00093;

    @(posedge clk); #1;
    $display(" PC=%0d | I-type imm (expected -5) = %0d", PC, $signed(imm));
    if ($signed(imm) === -5)
        $display(" [PASS] immGen I-type correct");
    else
        $display(" [FAIL] immGen I-type WRONG - got %0d", $signed(imm));

    $display("\n--- TEST 4: immGen S-type - SW x1,-4(x0) (32'hFE102E23) ---");
    inst = 32'hFE102E23;

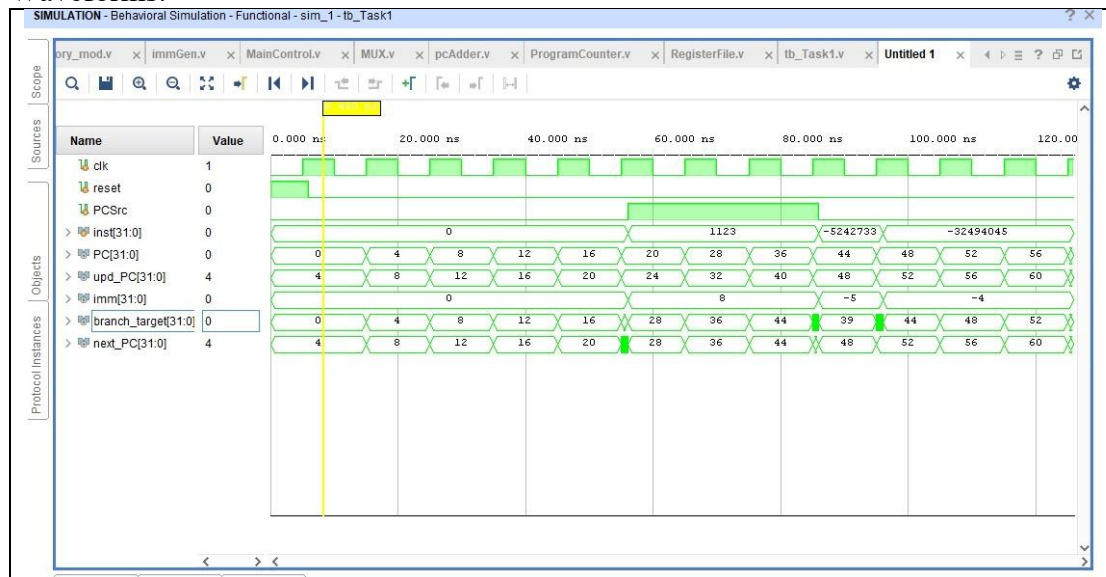
    @(posedge clk); #1;
    $display(" PC=%0d | S-type imm (expected -4) = %0d", PC, $signed(imm));
    if ($signed(imm) === -4)
        $display(" [PASS] immGen S-type correct");
    else
        $display(" [FAIL] immGen S-type WRONG - got %0d", $signed(imm));

    $display("\n--- All tests complete ---");
    #20;
    $finish;
end

endmodule
```



Waveforms:



Task 2

Design module code of **TopLevelProcessor** module:

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 04/19/2026 10:27:52 PM
// Design Name:
// Module Name: TopLevelProcessor
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////

module TopLevelProcessor(
    input clk,
    input reset
```



```
);  
wire [31:0] PC;  
wire [31:0] next_PC;  
wire [31:0] upd_PC;  
wire [31:0] branch_tar;  
  
wire [31:0] instruction;  
wire [6:0] opcode = instruction[6:0];  
wire [4:0] rd = instruction[11:7];  
wire [2:0] funct3 = instruction[14:12];  
wire [4:0] rs1 = instruction[19:15];  
wire [4:0] rs2 = instruction[24:20];  
wire [6:0] funct7 = instruction[31:25];  
wire regwrite, alusrc, memread, memwrite, memtoreg, branch;  
wire [1:0] aluop;  
wire [31:0] readData1, readData2;  
wire [31:0] imm;  
wire [31:0] ALU_B;  
wire [3:0] alucontrol;  
wire [31:0] ALUResult;  
wire zero;  
wire PCSrc;  
assign PCSrc = branch & zero;  
wire dataMemRead, dataMemWrite, LEDWrite, SwitchReadEnable;  
wire [31:0] memReadData;  
wire [31:0] writeBackData;  
ProgramCounter uPC (  
    .clk (clk),  
    .reset (reset),  
    .next_PC (next_PC),  
    .PC (PC)  
);  
instructionMemory uInstMem (  
    .instAddress (PC),  
    .instruction (instruction)  
);  
pcAdder uPCAdder (  
    .PC (PC),  
    .upd_PC (upd_PC)  
);  
immGen uImmGen (  
    .inst (instruction),  
    .imm (imm)  
);  
branchAdder uBranchAdder (  
    .PC (PC),  
    .imm (imm),  
    .branch_tar (branch_tar)
```



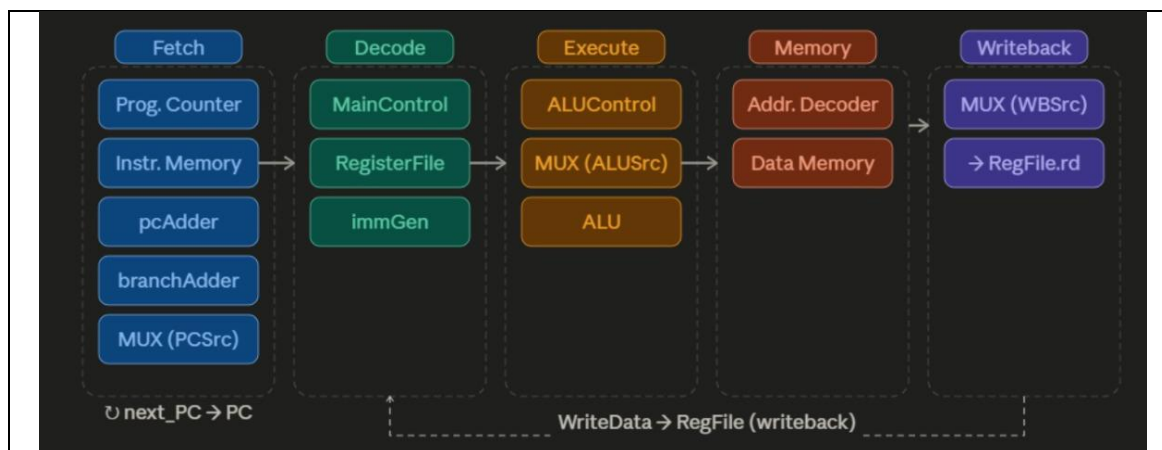

```
);  
MUX uPCMux (  
    .in0  (upd_PC),    // PCSrc=0 → PC+4  
    .in1  (branch_tar), // PCSrc=1 → branch target  
    .select (PCSrc),  
    .out  (next_PC)  
);  
MainControl uControl (  
    .opcode (opcode),  
    .regwrite (regwrite),  
    .alusrc (alusrc),  
    .memread (memread),  
    .memwrite (memwrite),  
    .memtoreg (memtoreg),  
    .branch (branch),  
    .aluop (aluop)  
);  
RegisterFile uRegFile (  
    .clk      (clk),  
    .rst      (reset),  
    .WriteEnable (regwrite),  
    .rs1      (rs1),  
    .rs2      (rs2),  
    .rd       (rd),  
    .WriteData (writeBackData),  
    .readData1 (readData1),  
    .readData2 (readData2)  
);  
MUX uALUSrcMux (  
    .in0  (readData2), // alusrc=0 → use rs2 (R-type)  
    .in1  (imm),       // alusrc=1 → use immediate (I/S-type)  
    .select (alusrc),  
    .out  (ALU_B)  
);  
ALUControl uALUCtrl (  
    .aluop (aluop),  
    .funct3 (funct3),  
    .funct7 (funct7),  
    .alucontrol (alucontrol)  
);  
ALU_module uALU (  
    .A      (readData1),  
    .B      (ALU_B),  
    .ALUControl (alucontrol),  
    .ALUResult (ALUResult),  
    .zero     (zero)  
);  
address_decoder uAddrDec (  

```

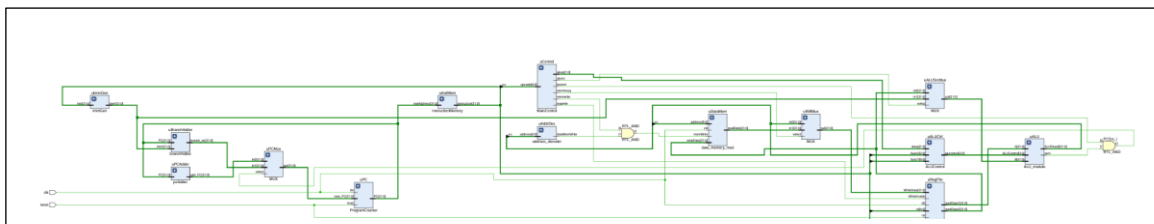


```
.address      (ALUResult[9:0]),  
.dataMemRead  (dataMemRead),  
.dataMemWrite (dataMemWrite),  
.LEDWrite     (LEDWrite),  
.SwitchReadEnable (SwitchReadEnable)  
);  
data_memory_mod uDataMem (  
  .clk      (clk),  
  .memWrite (memwrite & dataMemWrite),  
  .address  (ALUResult[9:0]),  
  .writeData (readData2),          // rs2 is the value stored  
  .readData  (memReadData)  
);  
MUX uWBMux (  
  .in0 (ALUResult), // memtoreg=0 → ALU result (R/I-type)  
  .in1 (memReadData), // memtoreg=1 → memory data (load)  
  .select (memtoreg),  
  .out (writeBackData)  
);  
endmodule
```

Datapath Diagram:



RTL Schematic:



Utilization Report:



task 2:

utilization report:

Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.

| Tool Version : Vivado v.2020.1 (win64) Build 2902540 Wed May 27 19:54:49
MDT 2020
| Date : Sun Apr 19 23:06:32 2026
| Host : DESKTOP-KTPK1OD running 64-bit major release (build 9200)
| Command : report_utilization -file TopLevelProcessor_utilization_synth.rpt -
pb TopLevelProcessor_utilization_synth.pb
| Design : TopLevelProcessor
| Device : 7a35tcpg236-1
Design State : Synthesized

Utilization Design Information

Table of Contents

-
1. Slice Logic
 - 1.1 Summary of Registers by Type
 2. Slice Logic Distribution
 3. Memory
 4. DSP
 5. IO and GT Specific
 6. Clocking
 7. Specific Feature
 8. Primitives
 9. Black Boxes
 10. Instantiated Netlists

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs	0	0	20800	0.00
LUT as Logic	0	0	20800	0.00
LUT as Memory	0	0	9600	0.00
Slice Registers	0	0	41600	0.00
Register as Flip Flop	0	0	41600	0.00
Register as Latch	0	0	41600	0.00
F7 Muxes	0	0	16300	0.00
F8 Muxes	0	0	8150	0.00



1.1 Summary of Registers by Type

	<i>Total</i>	<i>Clock Enable</i>	<i>Synchronous</i>	<i>Asynchronous</i>
0	-	-	-	-
0	-	-	Set	-
0	-	-	Reset	-
0	-	Set	-	-
0	-	Reset	-	-
0	Yes	-	-	-
0	Yes	-	Set	-
0	Yes	-	Reset	-
0	Yes	Set	-	-
0	Yes	Reset	-	-

2. Slice Logic Distribution

<i>Site Type</i>	<i>Used</i>	<i>Fixed</i>	<i>Available</i>	<i>Util%</i>
<i>Slice</i>	0	0	8150	0.00
<i>SLICEL</i>	0	0		
<i>SLICEM</i>	0	0		
<i>LUT as Logic</i>	0	0	20800	0.00
<i>LUT as Memory</i>	0	0	9600	0.00
<i>LUT as Distributed RAM</i>	0	0		
<i>LUT as Shift Register</i>	0	0		
<i>Slice Registers</i>	0	0	41600	0.00
<i>Register driven from within the Slice</i>	0			
<i>Register driven from outside the Slice</i>	0			
<i>Unique Control Sets</i>	0		8150	0.00

**** Note:** Available Control Sets calculated as Slice * 1, Review the Control Sets Report for more information regarding control sets.

3. Memory

<i>Site Type</i>	<i>Used</i>	<i>Fixed</i>	<i>Available</i>	<i>Util%</i>
------------------	-------------	--------------	------------------	--------------



	Used	Fixed	Available	Util%
Block RAM Tile	0	0	50	0.00
RAMB36/FIFO*	0	0	50	0.00
RAMB18	0	0	100	0.00

* Note: Each Block RAM Tile only has one FIFO logic available and therefore can accommodate only one FIFO36E1 or one FIFO18E1. However, if a FIFO18E1 occupies a Block RAM Tile, that tile can still accommodate a RAMB18E1

4. DSP

	Used	Fixed	Available	Util%
Site Type	Used	Fixed	Available	Util%
DSPs	0	0	90	0.00

5. IO and GT Specific

	Used	Fixed	Available	Util%
Site Type	Used	Fixed	Available	Util%
Bonded IOB	0	0	106	0.00
Bonded IPADs	0	0	10	0.00
Bonded OPADs	0	0	4	0.00
PHY_CONTROL	0	0	5	0.00
PHASER_REF	0	0	5	0.00
OUT_FIFO	0	0	20	0.00
IN_FIFO	0	0	20	0.00
IDELAYCTRL	0	0	5	0.00
IBUFDS	0	0	104	0.00
GTPE2_CHANNEL	0	0	2	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	20	0.00
PHASER_IN/PHASER_IN_PHY	0	0	20	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	250	0.00
IBUFDS_GTE2	0	0	2	0.00
ILOGIC	0	0	106	0.00
OLOGIC	0	0	106	0.00

6. Clocking



<i>Site Type</i>	<i>Used</i>	<i>Fixed</i>	<i>Available</i>	<i>Util%</i>
<i>BUFGCTRL</i>	0	0	32	0.00
<i>BUFIO</i>	0	0	20	0.00
<i>MMCME2_ADV</i>	0	0	5	0.00
<i>PLLE2_ADV</i>	0	0	5	0.00
<i>BUFMRCE</i>	0	0	10	0.00
<i>BUFHCE</i>	0	0	72	0.00
<i>BUFR</i>	0	0	20	0.00

7. Specific Feature

<i>Site Type</i>	<i>Used</i>	<i>Fixed</i>	<i>Available</i>	<i>Util%</i>
<i>BSCANE2</i>	0	0	4	0.00
<i>CAPTUREE2</i>	0	0	1	0.00
<i>DNA_PORT</i>	0	0	1	0.00
<i>EFUSE_USR</i>	0	0	1	0.00
<i>FRAME_ECCE2</i>	0	0	1	0.00
<i>ICAPE2</i>	0	0	2	0.00
<i>PCIE_2_1</i>	0	0	1	0.00
<i>STARTUPE2</i>	0	0	1	0.00
<i>XADC</i>	0	0	1	0.00

8. Primitives

<i>RefName</i>	<i>Used</i>	<i>Functional Category</i>
----------------	-------------	----------------------------

9. Black Boxes

<i>RefName</i>	<i>Used</i>
----------------	-------------

10. Instantiated Netlists



+-----+-----+	
<i>RefName</i> <i>Used</i>	
+-----+-----+	

Timing Analysis Report:

--



Lab 11: Design and FPGA Implementation of a Single-Cycle RISC-V Processor

Points Distribution

Task No.	OE2 (Design)	OE5 (Report)	OE6 (Reflection)	AR7
Task 1	/40	-	-	/20
Task 2	/20	/30		
Viva			/10	
Total Points	/80			/20
CLO Mapped	CLO 3			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.